LEZIONE 11 - Cicli: while e do-while

PARTE TEORICA (2h)

1. Concetto di Iterazione

L'**iterazione** (o ciclo/loop) è un costrutto fondamentale che permette di ripetere un blocco di istruzioni finché una determinata condizione rimane vera.

Perché sono importanti i cicli?

- Evitano la ripetizione di codice
- Permettono di processare grandi quantità di dati
- Consentono di eseguire operazioni fino al raggiungimento di un obiettivo

Elementi chiave di un ciclo:

1. **Inizializzazione**: preparare le variabili prima del ciclo
2. **Condizione**: espressione booleana che controlla se continuare
3. **Corpo**: blocco di istruzioni da ripetere
4. **Aggiornamento**: modifica delle variabili per avvicinarsi alla fine

---

2. Ciclo while

Il ciclo `while` ripete un blocco di codice finché la condizione è vera. La condizione viene verificata **prima** di ogni iterazione.

Sintassi:

```
while (condizione) {
    // corpo del ciclo
    // istruzioni da ripetere
}
```

Flowchart del while:



Esempio 1: Contare da 1 a 5

```c
#include <stdio.h>

int main() {
    int i = 1;  // Inizializzazione

    while (i <= 5) {  // Condizione
        printf("%d\n", i);
        i++;  // Aggiornamento
    }

    return 0;
}
```

Esempio 2: Somma di numeri inseriti dall’utente

```c
#include <stdio.h>

int main() {
    int numero;
    int somma = 0;

    printf("Inserisci numeri (0 per terminare):\n");
    scanf("%d", &numero);

    while (numero != 0) {
        somma += numero;
        printf("Somma parziale: %d\n", somma);
        scanf("%d", &numero);
    }

    printf("Somma totale: %d\n", somma);
    return 0;
}
```

---

3. Ciclo do-while

Il ciclo `do-while` esegue il corpo **almeno una volta**, poi controlla la condizione. La verifica avviene **dopo** ogni iterazione.

Sintassi:

```c
do {
    // corpo del ciclo
    // istruzioni da ripetere
} while (condizione);  // NOTA: il punto e virgola è obbligatorio!
```

Flowchart del do-while:



Esempio: Menu con validazione

```c
#include <stdio.h>

int main() {
    int scelta;

    do {
        printf("\n=== MENU ===\n");
        printf("1. Opzione A\n");
        printf("2. Opzione B\n");
        printf("3. Esci\n");
        printf("Scelta: ");
        scanf("%d", &scelta);

        switch(scelta) {
            case 1:
                printf("Hai scelto A\n");
                break;
```

```
            case 2:
                printf("Hai scelto B\n");
                break;
            case 3:
                printf("Arrivederci!\n");
                break;
            default:
                printf("Scelta non valida!\n");
        }
    } while (scelta != 3);

    return 0;
}
```

---

4. Differenze tra while e do-while

Caratteristica	while	do-while
Controllo condizione	Prima del corpo	Dopo il corpo
Esecuzioni minime	0 (può non eseguire mai)	1 (esegue sempre almeno una volta)
Uso tipico	Quando non si sa se eseguire	Quando serve almeno un'esecuzione

Esempio comparativo:

```
// while - potrebbe non stampare nulla
int x = 10;
while (x < 5) {
    printf("Questo non viene mai stampato\n");
    x++;
}
```

```c
// do-while - stampa almeno una volta
int y = 10;
do {
    printf("Questo viene stampato una volta: %d\n", y);
    y++;
} while (y < 5);
```

Quando usare do-while:

- Menu interattivi
- Validazione input (richiedi finché non è valido)
- Quando serve almeno un'iterazione garantita

---

5. Cicli Infiniti e Condizioni di Uscita

Ciclo infinito (da evitare accidentalmente!):

```c
// ERRORE: condizione sempre vera
int i = 1;
while (i > 0) {
    printf("Loop infinito!\n");
    // Manca i++ o una condizione di uscita
}
```

Ciclo infinito intenzionale:

```c
// OK se gestito con break
while (1) {  // o while(1) sempre vero
    int input;
    printf("Inserisci numero (0 per uscire): ");
    scanf("%d", &input);

    if (input == 0) {
```

```
        break;  // Esce dal ciclo
    }

    printf("Hai inserito: %d\n", input);
}
```

Errori comuni:

```
// ERRORE 1: dimenticare l'aggiornamento
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    // Manca i++! Ciclo infinito
}

// ERRORE 2: punto e virgola dopo while
int i = 0;
while (i < 10);  // ERRORE: corpo vuoto!
{
    printf("%d\n", i);
    i++;
}

// ERRORE 3: condizione sbagliata
int i = 0;
while (i = 5) {  // ERRORE: assegnamento invece di confronto!
    printf("%d\n", i);
    i++;
}
```

PARTE PRATICA (2h)

Esercizio 1: Somma di numeri fino a condizione

```c
#include <stdio.h>

int main() {
    int numero;
    int somma = 0;
    int contatore = 0;

    printf("Inserisci numeri positivi (numero negativo per terminare):\n");

    scanf("%d", &numero);
    while (numero >= 0) {
        somma += numero;
        contatore++;
        printf("Inserisci un altro numero: ");
        scanf("%d", &numero);
    }

    if (contatore > 0) {
        printf("\nSomma: %d\n", somma);
        printf("Media: %.2f\n", (float)somma / contatore);
    } else {
        printf("Nessun numero inserito.\n");
    }

    return 0;
}
```

Compito: Modifica il programma per trovare anche il massimo tra i numeri inseriti.

Esercizio 2: Validazione input con cicli

```c
#include <stdio.h>

int main() {
    int eta;

    // Validazione con do-while
    do {
        printf("Inserisci la tua età (0-120): ");
        scanf("%d", &eta);

        if (eta < 0 || eta > 120) {
            printf("Età non valida! Riprova.\n");
        }
    } while (eta < 0 || eta > 120);

    printf("Età valida: %d anni\n", eta);

    // Classificazione
    if (eta < 18) {
        printf("Sei minorenne\n");
    } else if (eta < 65) {
        printf("Sei adulto\n");
    } else {
        printf("Sei senior\n");
    }

    return 0;
}
```

Esercizio 3: Gioco “Indovina il numero”

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int numeroSegreto, tentativo, tentativi = 0;

    // Inizializza il generatore di numeri casuali
    srand(time(NULL));
    numeroSegreto = rand() % 100 + 1;  // Numero tra 1 e 100

    printf("=== INDOVINA IL NUMERO ===\n");
    printf("Ho pensato un numero tra 1 e 100\n\n");

    do {
        printf("Il tuo tentativo: ");
        scanf("%d", &tentativo);
        tentativi++;

        if (tentativo < numeroSegreto) {
            printf("Troppo basso! Riprova.\n");
        } else if (tentativo > numeroSegreto) {
            printf("Troppo alto! Riprova.\n");
        } else {
            printf("\n🎉 COMPLIMENTI! Hai indovinato in %d tentativi!\n",
                tentativi);
        }
    } while (tentativo != numeroSegreto);

    return 0;
}
```

Esercizio 4: Calcolo del fattoriale

```c
#include <stdio.h>

int main() {
    int n, i;
    long long fattoriale = 1;

    printf("Inserisci un numero: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Il fattoriale non è definito per numeri negativi!\n");
    } else {
        // Versione con while
        i = 1;
        while (i <= n) {
            fattoriale *= i;
            i++;
        }

        printf("%d! = %lld\n", n, fattoriale);
    }

    return 0;
}
```

Compito: Riscrivi il programma usando un ciclo do-while.

Esercizi da svolgere autonomamente:

1. **Contatore alla rovescia**: Scrivi un programma che conta da 10 a 0 usando un ciclo while

2. **Tabellina**: Chiedi all'utente un numero e stampa la sua tabellina (da 1 a 10) usando do-while

3. **Somma pari e dispari**: Inserisci numeri fino a 0, poi stampa la somma dei numeri pari e quella dei dispari separatamente

4. **Password**: Crea un sistema che chiede una password e permette massimo 3 tentativi usando do-while

5. **Sequenza Fibonacci**: Stampa i primi N numeri della sequenza di Fibonacci usando while

6. **MCD (Massimo Comun Divisore)**: Calcola l'MCD di due numeri usando l'algoritmo di Euclide con un ciclo while

---

Riepilogo Concetti Chiave

✓ Il **while** controlla la condizione prima di eseguire il corpo
✓ Il **do-while** esegue il corpo almeno una volta, poi controlla
✓ Ogni ciclo necessita di inizializzazione, condizione e aggiornamento
✓ Attenzione ai cicli infiniti accidentali
✓ do-while è ideale per menu e validazione input

Prossima lezione: Approfondiremo il ciclo **for** e i cicli annidati!